	Carátula para entrega de prácticas
Facultad de Ingeniería	Laboratorios de docencia

Laboratorio de Computación Salas A y B

Profesor(a): Rene Adrián Davila Pérez

Asignatura: Programación Orientada a Objetos

Grupo: 7

No de Práctica(s): 10

Integrante(s): 323001315, 426057857, 323008754, 426058665, 323067267

No. de lista o brigada: 5

Semestre: 2027-1

Fecha de entrega: 6 de Noviembre

Observaciones:

CALIFICACIÓN: _____

Índice

1. Introducción **2**
1.1. Planteamiento del problema 2
1.2. Motivación 2
1.3. Objetivos 2

2. Marco Teórico **3**
2.1. Errores de Ejecución y Excepciones 3
2.2. Manejo de Excepciones (`try-catch-finally`) 3
2.3. Propagación y Relanzamiento de Excepciones 3
2.4. Excepciones Propias y su aplicación en Interfaz Gráfica (GUI) 4

3. Desarrollo **4**
3.1. Ejercicio 1 4
3.2. Ejercicio 2 5

4. Resultados **7**
4.1. Ejercicio 1- Consultar con IA Gen: excepciones más comunes en GUI 7
4.2. Ejercicio 2 8
4.2.1. Resultados de la Aplicación CLI 8
4.3. Resultados de la Aplicación GUI 9
4.4. Resultados de la Forma 1 de Relanzamiento (CLI) 10

5. Conclusiones **10**

1. Introducción

1.1. Planteamiento del problema

El problema a resolver consiste en implementar y demostrar el correcto manejo y propagación de errores en tiempo de ejecución en Java, centrándose específicamente en la técnica de relanzamiento de excepciones. Se requiere construir componentes de software que capturen excepciones estándar, como `NumberFormatException` al procesar texto o `IOException` al leer datos, en métodos de bajo nivel. Posteriormente, el sistema debe delegar el tratamiento de estos fallos a las capas superiores de la arquitectura mediante el relanzamiento, ya sea enviando la misma excepción o encapsulándola en una excepción propia (como `DatoInvalidoException` o `ProcesoException`). Esta mecánica debe ilustrarse resolviendo dos escenarios precisos: una aplicación de interfaz de línea de comandos (CLI) y una de interfaz gráfica de usuario (GUI).

1.2. Motivación

En el desarrollo de software robusto, la detección y el manejo de errores es la parte más crítica; estadísticamente, el 80 % de las fallas de una aplicación son producidas por el 20 % del código. Un error de ejecución no controlado provoca que la aplicación termine de forma abrupta. Es imperativo implementar el relanzamiento de excepciones porque, frecuentemente, el método donde ocurre el error original tiene la capacidad de reaccionar localmente (por ejemplo, registrando el evento en una bitácora) pero carece del contexto adecuado para resolver la falla de cara al usuario. Relanzar las excepciones permite separar la lógica de negocio de la lógica de presentación, garantizando que las capas visuales (como los botones de una ventana) sean las encargadas finales de atrapar el error y mostrar cuadros de diálogo amigables, evitando que el sistema colapse.

1.3. Objetivos

Lo que se espera obtener al darle solución al problema es:

- Identificar bloques de código propensos a generar errores y aplicar el bloque `try-catch-throws` para el manejo de situaciones excepcionales en tiempo de ejecución.
- Construir una aplicación CLI en Java que ilustre el flujo de relanzamiento, capturando una excepción y delegándola hasta el método `main`.
- Implementar una aplicación GUI (basada en `JFrame`) que capture excepciones comunes originadas en componentes gráficos y lance una excepción propia (`DatoInvalidoException`) para advertir al usuario mediante cuadros de diálogo.

- Diferenciar de manera práctica entre relanzar la misma excepción capturada (reenviar el mismo objeto) y encapsular el error lanzando una nueva excepción marcada.

2. Marco Teórico

2.1. Errores de Ejecución y Excepciones

Una aplicación puede presentar errores sintácticos, semánticos y de ejecución [1].

Los errores en tiempo de ejecución provocan que la aplicación termine de manera abrupta al presentarse situaciones no previstas, como accesos a localidades no reservadas o formato incorrecto de datos [1].

Para controlar esto, Java utiliza **excepciones**, que son objetos derivados de la clase `Throwable` que representan una condición excepcional que altera el flujo normal del programa [1, 2].

Las excepciones en Java se dividen en dos grandes grupos:

- **Excepciones marcadas:** Son aquellas cuya captura es obligatoria por el compilador, ya que derivan directamente de la clase `Exception` [1].
- **Excepciones no marcadas:** Son excepciones en tiempo de ejecución (subclases de `RuntimeException`, como `NumberFormatException`) cuya captura no es estrictamente obligatoria, pero sí recomendada para evitar cierres inesperados [1].

2.2. Manejo de Excepciones (`try-catch-finally`)

El mecanismo principal para controlar las excepciones se compone de tres bloques clave:

- El bloque `try` se utiliza para definir la porción de código en la cual es susceptible que ocurra una excepción [1].
- El bloque `catch` se emplea para capturar y definir el manejo específico de la excepción generada [1].
- El bloque `finally` define instrucciones que se ejecutarán de manera independiente al resultado de los bloques anteriores, ocurra o no un error [1].

2.3. Propagación y Relanzamiento de Excepciones

No es obligatorio tratar por completo las excepciones en el método exacto donde ocurren; es posible delegar esta responsabilidad a métodos superiores en la pila de llamadas [1].

- **Propagación (`throws`):** Se utiliza en la firma del método para indicar explícitamente que dicho método puede arrojar una o varias excepciones marcadas, obligando a quien lo invoque a manejarlas [2].
- **Relanzamiento (`throw`):** Consiste en capturar la excepción con `catch`, realizar una acción local (como registrar el evento en bitácora o limpiar recursos) y volver a lanzarla para que otro método de nivel superior la atienda de manera definitiva [2].

El relanzamiento se puede efectuar de dos formas principales:

1. **Relanzar la misma excepción:** Se utiliza la instrucción `throw e;` dentro del `catch`, lo que reenvía exactamente el mismo objeto de excepción capturado hacia el método invocador [2].
2. **Relanzar una excepción propia:** Se captura una excepción original y, en su lugar, se lanza una nueva instancia de una excepción definida por el programador (por ejemplo, `throw new ExcepcionPropia(...)`) [3].

2.4. Excepciones Propias y su aplicación en Interfaz Gráfica (GUI)

Java permite crear clases que se comporten como excepciones personalizadas a la medida del negocio, para lo cual solo es necesario heredar de la clase `Exception` (haciéndolas marcadas) [1].

La técnica de relanzamiento y de excepciones propias cobra especial relevancia en arquitecturas con Interfaz Gráfica de Usuario (GUI). En estas aplicaciones, los métodos de procesamiento lógico o acceso a datos atrapan errores técnicos de formato, pero al carecer de mecanismos visuales para notificarlos, los relanzan hacia las capas superiores [3]. De este modo, es el controlador de la vista (como el evento de un botón) el encargado final de atrapar la excepción definitiva y desplegar un cuadro de diálogo informativo al usuario (como un `JOptionPane`), preservando la ejecución del subproceso de la interfaz [2].

3. Desarrollo

3.1. Ejercicio 1

1. **Inicio de la actividad.** Se parte del ejercicio asignado en la práctica, el cual consiste en consultar a una Inteligencia Artificial cuáles son las excepciones más comunes en las Interfaces Gráficas de Usuario (GUI).
2. **Patrones universales en GUI.** Como resultado de la consulta, la IA explica que las excepciones varían según el lenguaje o framework (Java Swing, C# WPF, Python PyQt, etc.), pero comparten patrones universales debido a la arquitectura basada en eventos que utilizan casi todos los sistemas visuales.

3. **Violación de subprocesos cruzados (Cross-Thread Operation** / `IllegalThread StateException`**).** Ocurre cuando se intenta modificar un componente visual (como cambiar el texto de una etiqueta) desde un hilo de fondo o secundario. En casi todos los frameworks, la interfaz de usuario solo puede y debe ser actualizada desde su hilo principal (conocido como *UI Thread* o *Event Dispatch Thread*). Es el error de GUI más común.

4. **Desbordamiento de pila por eventos (**`StackOverflowError` / **Infinite Event Loop).** Se produce cuando los manejadores de eventos (*event listeners*) entran en una reacción en cadena infinita. Por ejemplo: el evento "al cambiar valor" del Componente A actualiza el Componente B, y el evento del Componente B vuelve a actualizar el Componente A, saturando la memoria instantáneamente.

5. **Acceso a objeto destruido (**`ObjectDisposedException` / `WidgetDestroyed Error`**).** Se lanza cuando una tarea asíncrona o un temporizador intenta actualizar, leer o interactuar con una ventana, cuadro de diálogo o control gráfico que el usuario ya ha cerrado o que el sistema ya ha liberado de la memoria.

6. **Referencia nula (**`NullReferenceException` / `NullPointerException`**).** Sucede al intentar configurar las propiedades de un elemento visual (colores, texto, tamaño) antes de que haya sido instanciado o renderizado en el ciclo de vida de la aplicación, o al buscar un componente en el árbol visual que no existe.

7. **Estado u operación inválida (**`InvalidOperationException` / `IllegalArgument Exception`**).** Ocurre al intentar operaciones lógicamente imposibles en la jerarquía visual. Los ejemplos clásicos incluyen intentar añadir un elemento como hijo de sí mismo, asignar dimensiones negativas a una ventana, o intentar modificar una colección de elementos visuales (como los ítems de un `ComboBox`) mientras otro proceso la está iterando.

3.2. Ejercicio 2

1. **Inicio del trabajo.** Se parte del tema asignado en la práctica, el relanzamiento de excepciones.

2. **Diagrama UML de clases.** Se presenta la jerarquía `Exception`, `Runtime Exception`, `IllegalArgumentException` y `NumberFormatException`. Se agregan `DatoInvalidoException`, que hereda de `Exception`, y `VentanaEdad`, con el atributo `campoEdad` y los métodos `leerEdad` y `guardar`. Se marcan con flechas la herencia y las dependencias de lanzar (*«throws»*) y capturar (*«catch»*).

3. **Excepciones marcadas y no marcadas.** Se explica que `Exception` y `DatoInvalido Exception` son marcadas, por lo que hay que declararlas con `throws`. `Runtime Exception` y sus subclases, como `NumberFormatException`, son no marcadas.

4. **Diagrama de flujo.** Se muestra el recorrido del relanzamiento: `main()` llama a `procesar()`, que llama a `leerDato()`. Ahí se lanza una `IOException`, `procesar()` la captura, la registra y la relanza como `ProcesoException`, y `main()` la atiende para terminar el programa.

5. **Pseudocódigo.** Se presenta el algoritmo `leerEdad`, que intenta convertir el texto a entero. Si falla, captura el error, escribe *«Error: dato inválido»* y relanza, ya sea la misma excepción o una propia. Se presenta también el algoritmo del botón `Guardar`, que atrapa la excepción y muestra un diálogo con el mensaje.

6. **Flujo con la entrada "abc".** Se explica el caso con un ejemplo: la conversión a entero falla, `leerEdad` captura y relanza, y el botón atrapa la excepción y muestra el mensaje al usuario.

7. **Qué es relanzar una excepción.** Se define como capturar la excepción con `catch`, hacer algo con ella y volver a lanzarla para que otro método la atienda. Se explica en tres pasos: `try` (ocurre el error), `catch` (se captura y se hace algo local, como registrar o limpiar) y `throw` (se lanza de nuevo para que el método que llamó decida qué hacer).

8. **Forma 1: relanzar la misma excepción.** Se muestra el método `leerEdad` con `Integer.parseInt`. El `catch` de `NumberFormatException` imprime el error y ejecuta `throw e;`, que envía el mismo objeto al método que llamó. Como la excepción es no marcada, no hace falta `throws`.

9. **Forma 2: relanzar una excepción propia.** Primero se muestra la clase `DatoInvalidoException`, que extiende `Exception` y pasa el mensaje a `super(mensaje)`. Después se muestra `leerEdad` con `throws DatoInvalidoException`, que en el `catch` lanza `throw new DatoInvalidoException("La edad debe ser un número");`.

10. **Ejemplo en una ventana: botón Guardar.** Se presenta el caso en que el usuario escribe "abc" en el campo de edad. `getText()` devuelve "abc", `parseInt` lanza `NumberFormatException`, el `catch` de `leerEdad` la captura y lanza `DatoInvalidoException`, la excepción sube hasta el botón, y su `catch` muestra el mensaje con `JOptionPane.showMessageDialog`.

11. **Cierre del trabajo.** Se confirma que la presentación incluye el diagrama de clases, el diagrama de flujo, el pseudocódigo, las dos formas de relanzar, el ejemplo en la ventana y el resumen, y se realiza la exposición.

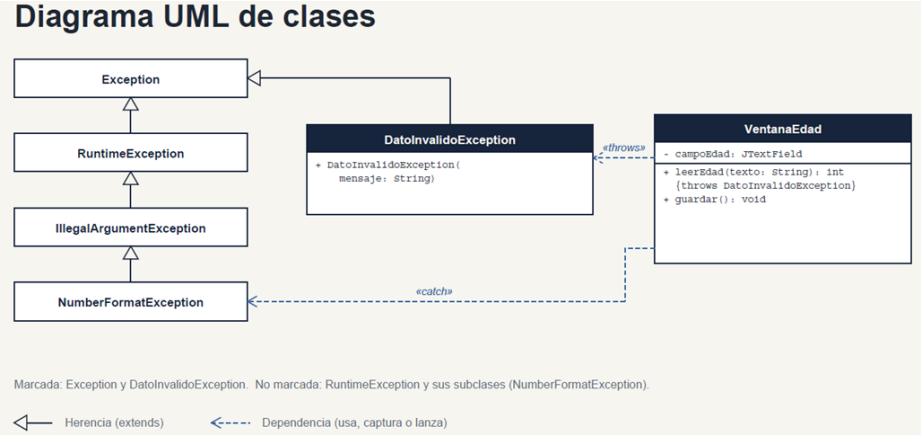


Figura 1: Diagrama UML



Figura 2: Diagrama de flujo

4. Resultados

4.1. Ejercicio 1- Consultar con IA Gen: excepciones más comunes en GUI

Las excepciones en el desarrollo de Interfaces Gráficas de Usuario (GUI) varían según el lenguaje o framework (Java Swing, C# WPF, Python PyQt, etc.), pero comparten patrones universales debido a la arquitectura basada en eventos que utilizan casi todos los sistemas visuales.

- **Violación de subprocesos cruzados (Cross-Thread Operation** / `IllegalThreadStateException`**):** Ocurre cuando se intenta modificar un componente visual (como cambiar el texto de una etiqueta) desde un hilo de fondo o secundario.

En casi todos los frameworks, la interfaz de usuario solo puede y debe ser actualizada desde su hilo principal (conocido como *UI Thread* o *Event Dispatch Thread*). Es el error de GUI más común.

- **Desbordamiento de pila por eventos (`StackOverflowError` / Infinite Event Loop):** Se produce cuando los manejadores de eventos (*event listeners*) entran en una reacción en cadena infinita. Por ejemplo: el evento "al cambiar valor" del Componente A actualiza el Componente B, y el evento del Componente B vuelve a actualizar el Componente A, saturando la memoria instantáneamente.

- **Acceso a objeto destruido (`ObjectDisposedException` / `WidgetDestroyed Error`):** Se lanza cuando una tarea asíncrona o un temporizador intenta actualizar, leer o interactuar con una ventana, cuadro de diálogo o control gráfico que el usuario ya ha cerrado o que el sistema ya ha liberado de la memoria.

- **Referencia nula (`NullReferenceException` / `NullPointerException`):** Sucede al intentar configurar las propiedades de un elemento visual (colores, texto, tamaño) antes de que haya sido instanciado o renderizado en el ciclo de vida de la aplicación, o al buscar un componente en el árbol visual que no existe.

- **Estado u operación inválida (`InvalidOperationException` / `IllegalArgument Exception`):** Ocurre al intentar operaciones lógicamente imposibles en la jerarquía visual. Los ejemplos clásicos incluyen intentar añadir un elemento como hijo de sí mismo, asignar dimensiones negativas a una ventana, o intentar modificar una colección de elementos visuales (como los ítems de un `ComboBox`) mientras otro proceso la está iterando.

4.2. Ejercicio 2

4.2.1. Resultados de la Aplicación CLI

Al ejecutar la clase `FlujoRelanzamiento`, se observa en la consola el recorrido de la excepción. Primero, el método `procesar()` intercepta la `IOException` original y la registra. Posteriormente, la encapsula y relanza como `ProcesoException`, la cual es finalmente atrapada y atendida por el método `main()`, mostrando la causa original del fallo sin que el programa colapse.

```
PS C:\Users\acer\Downloads\files> java FlujoRelanzamiento
procesar(): registrando -> No se pudo leer el archivo
main(): atendiendo -> Falló el procesamiento
Causa original: No se pudo leer el archivo
Fin del programa
PS C:\Users\acer\Downloads\files>
```

Figura 3: Ejecución FlujoRelanzamiento.

4.3. Resultados de la Aplicación GUI

Al ejecutar la clase `VentanaEdad`, se despliega la interfaz con el campo de texto. Si el usuario ingresa un dato inválido (como la letra "abc") y presiona el botón, la aplicación no se cierra abruptamente. En su lugar, el error de formato es capturado internamente y relanzado como una `DatoInvalidoException`, lo que resulta en la aparición de un cuadro de diálogo nativo advirtiendo al usuario del error.

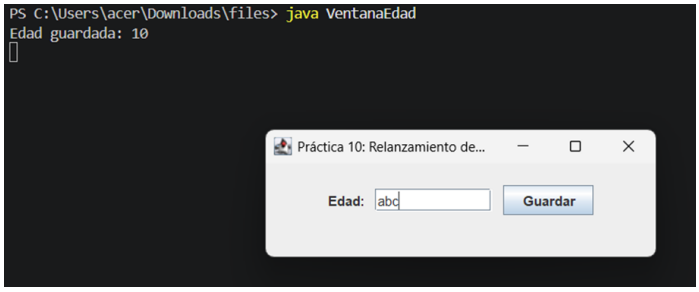


Figura 4

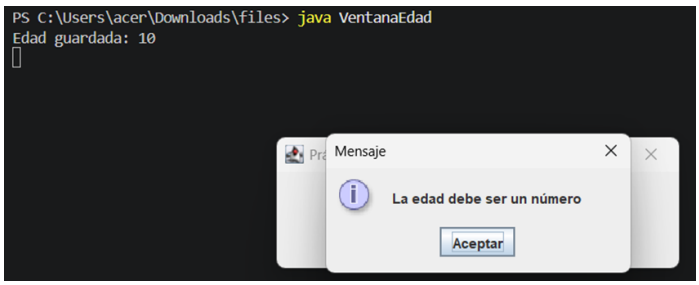


Figura 5

Por otro lado, si el usuario ingresa un valor numérico correcto (por ejemplo, "10"), el flujo se ejecuta con normalidad, validando el dato y confirmando la acción exitosa mediante otro cuadro de diálogo, demostrando la estabilidad de la ventana.

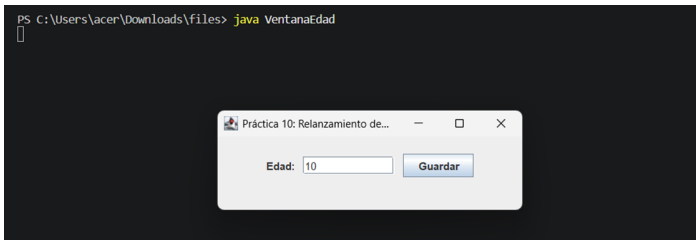


Figura 6

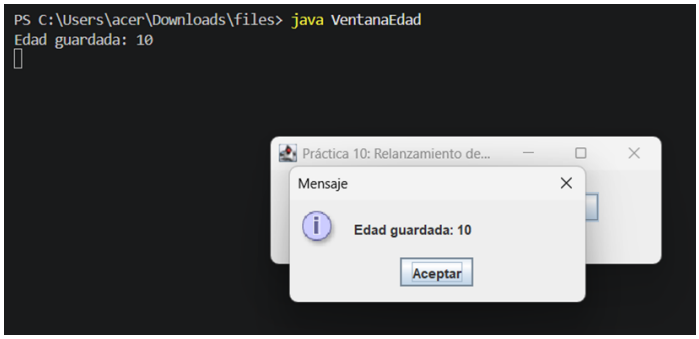


Figura 7

4.4. Resultados de la Forma 1 de Relanzamiento (CLI)

Al ejecutar el programa de demostración de la **Forma 1** (Figura 8), el método `leerEdad()` captura la excepción no marcada `NumberFormatException`, registra el error localmente (*"Error: dato inválido"*) y relanza exactamente el mismo objeto con la instrucción `throw e;`. Al ser una excepción no marcada, el método no requiere la cláusula `throws` en su firma. Finalmente, `main()` atiende la excepción y muestra el mensaje original de Java (*For input string: "abc"*), lo que confirma que se reenvió la misma excepción capturada.

```
PS C:\Users\acer\Downloads\files> java Forma1Demo
Error: dato inválido
main() atendió la excepción: For input string: "abc"
PS C:\Users\acer\Downloads\files>
```

Figura 8: Ejecución en consola de la Forma 1 Demo.

5. Conclusiones

Los resultados confirman que el relanzamiento de excepciones permite separar el tratamiento técnico de un error de su notificación al usuario. En la **Forma 1** (`Forma1Demo`), `leerEdad()` capturó la `NumberFormatException`, la registró y la relanzó con `throw e;` sin declarar `throws`, por ser no marcada; `main()` la atendió y mostró el mensaje original de Java. En la **Forma 2** (`VentanaEdad`), `leerEdad()` convirtió esa excepción en una `DatoInvalidoException`, marcada, mediante `throw new`, por lo que tuvo que declarar `throws`; el botón Guardar la atrapó y mostró un mensaje claro con `JOptionPane`.

La comparación de ambas formas permite distinguir su alcance: la primera reenvía el mismo objeto y conserva el mensaje técnico, mientras que la segunda crea una excepción propia con un mensaje adecuado para el usuario. De igual modo, en `FlujoRelanzamiento`, `procesar()` encapsuló la `IOException` en una `ProcesoException`

y `main()` la atendió sin que el programa terminara de forma abrupta. Así, cada método realiza solo la acción que le corresponde y la capa final atrapa la excepción definitiva, lo que cumple los objetivos de la práctica.

Referencias

[1] Facultad de Ingeniería UNAM, *Manual de prácticas del Laboratorio de Programación orientada a objetos (MADO-22)*, Guía práctica de estudio 10: Excepciones y errores. 2017.

[2] P. Deitel y H. Deitel, *Cómo programar en Java*, 7ma ed. México: Pearson Educación, 2008.

[3] Oracle, *The Java Tutorials: Exceptions*, Oracle Documentation. [En línea]. Disponible: https://docs.oracle.com/javase/tutorial/essential/exceptions/